

2053

SupNum

Devoir de Structures de données et complexité S3 2023-2024

Documents non autorisés !

Durée : 90 minutes

Exercice 1 : Soit la fonction suivante (4 points)

```

void fonc(int n){
    for(int i=1; i<=n;i++){
        for(int j=1; j<=i;j++){
            cout<<i<<","<<j<<endl;
        }
    }
}

```

- Qu'affiche la fonction avec $n=5$?
- Combien de fois, en fonction de n , l'instruction `cout` est exécutée ?

Exercice 2 : Qu'affiche le code suivant ? (4 points)

```

#include <iostream>
using namespace std;
typedef struct etudiant;
struct etudiant{
    int matricule;
    char * nom;
    char * prenom;
    int genre;
    int niv;
};
int main() {
    etudiant e1 = {22001, "Ahmed", "Fatma", 0,2};
    etudiant e2 = {21001, "Cheikh", "Ahmed", 1,2};
    etudiant * ptre1 = &e1; etudiant * ptre2 = &e2;
    etudiant e_tab[2]={e1,e2};
    cout<<"["<<e1.matricule<<","<<e1.nom<<","<<e1.prenom<<","<<e1.genre<<","<<e1.niv<<"]\n";
    ptre1->genre = 1; ptre1->prenom = "Sidi"; ptre2->genre = 0; ptre2->prenom = "Mariem";
    for (int i = 0; i<2; i++){
        cout<<"["<<e_tab[i].nom<<","<<e_tab[i].prenom<<","<<e_tab[i].genre<<"]\n";
    }
    cout<<"["<<(*ptre1).prenom<<","<<(*ptre2).prenom<<","<<ptre1->prenom<<","<<ptre2->prenom<<"]\n";
    return 0;
}

```

Exercice 3 : Listes chaînées (LC) (5 points)

- Rappeler la structure de données pour représenter une liste chaînée dont les éléments sont des entiers.
- Écrire une fonction C++ ou un pseudocode qui calcule la somme des éléments d'une LC.
- Supposons que nous volons garder les éléments triés selon l'ordre croissant. Écrire une fonction C++ ou un pseudocode qui permet d'ajouter un élément à sa bonne position dans une LC déjà triée.

Exercice 4 : (5 points)

En considérant la structure **etudiant** dans l'exercice 2, on souhaite manipuler une Pile dont les éléments sont des étudiants.

- Rappeler la structure des piles vue en cours (on considère que ses éléments sont de type **etudiant** et non pas des entiers), en rappelant la différence théorique avec la structure file.
- Écrire un algorithme qui prend comme paramètres deux piles : **p1** supposée remplie d'étudiants et **p2** supposée vide. L'algorithme met les garçons (genre =1) dans la pile **p2** selon leur ordre dans **p1** et laisse les filles dans **p1** sans modifier leur ordre dans **p1**.

Exercice 5 : problème du match de football (5 points)

Afin de mettre en place une application permettant de jouer au football entre deux équipes, on va voir comment un match de football peut être traité en utilisant les techniques d'algorithmique vues en cours. En effet, les règles suivantes représentent une première modélisation simplifiée de ce problème :

- Un joueur est une structure qui contient les informations nécessaires (Son numéro, son équipe (0 ou 1), sa distance de son gardien (un entier positif), sa position (G : gardien, M : Milieu, D : défenseur, A : Attaquant). Pour simplifier, on suppose qu'entre chaque deux positions du tableau, il y a 5 mètres de distance)
- La répartition des joueurs en stade est représentée par un tableau de listes chaînées comme suit :

1	2 ...	i	21	22
Numerojour : 23 Position : G Equipe : 1 Distance de son gardien : 0		Numerojour : 10, Position : M Equipe : 0, Distance : 15 ↓ Numerojour : 5, Position : D Equipe : 1, Distance : 90		Numerojour : 1 Position : G Equipe : 0 Distance de son gardien : 0

- Le match est une structure qui contient les informations suivantes : le tableau de listes chaînées de joueurs, le joueur qui a le ballon, le joueur qui le lui a passé, le score, le temps restant. Le tableau ci-dessous avec le tableau de jour ci-dessus, représente un exemple d'un match en cours :

Joueur en possession du ballon	Joueur qui avait le ballon	ButEquipe1	ButEquipe2	Temps restant (0 à 90)
Structure qui contient les informations du joueur en possession du ballon	Par exemple le premier dans liste à la position i (Numéro 10 de l'équipe 0)	1	0	5

- a. Écrire la structure représentant un joueur.
- b. Écrire la structure représentant le match en instant t (tableaux de joueurs, les autres informations sur le match).
- c. On considère l'algorithme *SeDeplacer*(match m , joueur j , int nombreDeCaseDedecalage). Par exemple *SeDeplacer*(m , {3, 'D', 1, 5}, 3) veut dire que le joueur numéro 3 qui est dans la liste à la position 2 (distance= 5 et équipe =1) va se déplacer à la position 5 dans le tableau. Le déplacement peut être négatif. Écrire cet algorithme en modifiant la position et la distance du joueur de son gardien. Notez que le gardien ne se déplace pas. Indicateurs : il faut trouver la case où se trouve le joueur à partir de sa distance de son gardien, ensuite, il faut le trouver dans la liste chaînée correspondant, puis, il faut l'enlever de cette liste et le mettre dans sa nouvelle position avec les modifications sur la distance.
- d. **Situation hors-jeu** : cette situation est signalée lorsque le joueur qui a le ballon l'a reçu de l'un de ses co-équipiers et qu'il est plus proche ou de même distance du gardien de l'adversaire que tous les joueurs de l'autre équipe. Écrire un algorithme *horsjeu*(match m) qui vérifie si la situation d'hors-jeu doit être signalée.
- e. **SifflerUnbut** : Un but est marqué lorsque le ballon est avec un joueur qui n'est pas en position d'hors-jeu et qu'il est plus proche du gardien de l'adversaire que tous les joueurs de l'autre équipe.
Écrire l'algorithme *sifflerUnbut*(match m) qui reçoit en paramètres un match, vérifie s'il y a un but à siffler et modifie le score s'il y a un but. Puis, il modifie le joueur qui a le ballon pour être le gardien qui a reçu le but.

Bonne réussite !